Reflection Paper: Problem #4
Learn to Read Other People's Code

Nick Galus

The first new idea that stood out to me was the concept of reading code like an archaeologist. The article by William Shawn framed it that way and I thought it was a genuinely useful way to think about it. When you jump into an unfamiliar codebase you are not just reading logic, you are trying to reconstruct context. Tools like git blame and git log let you see who wrote what, when things changed, and what problems the team was fighting at the time. I had used version control before but I never thought of the commit history as something to read for understanding. That reframe was new to me.

The second idea that was new to me came from the Skorkin article. He suggested that if you are really stuck on a piece of code you should try refactoring it as a way to understand it. I had always thought of refactoring as something you do after you understand the code, not as a tool for getting there. The logic makes sense though. When you pull a function apart and reorganize it you are forced to engage with what it is actually doing instead of just staring at it. I have definitely been the person scrolling up and down through unfamiliar code with a confused expression, and this gives me something more active to do instead.

The third idea was from the Coleman article about tracing chains of actions backward from a known output. I had always tried to read other people's code top to bottom starting from main, which usually leaves me overwhelmed pretty fast. Starting from something you know the code does and working backward is a much more manageable entry point. It is similar to how I debug my own code when something breaks and I already know where the bad output is showing up.

(A)gree. It is important to learn to read other people's code and I do not think it is even close. At my internship at Polaris this summer I am not going to be handed a blank file and told to start from scratch. I am going to be dropped into an existing embedded software codebase and expected to contribute to it. The same is true for basically every real job in software. Even in the Cyber City project I have had to read through OpenPLC source behavior and understand how the ST compiler handles located variables just to debug issues in my own code. If I could only work with code I personally wrote I would be nearly useless in a professional setting. The articles make a fair point that reading code feels boring and hard compared to writing it, and I will not pretend otherwise. But the payoff is too real to ignore. The more code you read the

faster you get at it and the better your own code gets as a side effect. That is a good deal.